

①. Process Synchronization

* Background:

- Concurrent access to shared data may result in data inconsistency.
- Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes.
- Suppose we want to provide a solution to the consumer-producer problem that fills all the buffers we can do so by having integer count that keeps track of the number of full buffers. First the count is 0. It is incremented by producer after it produces new buffer. & decremented by the consumer after it consumes the buffer.

Producers: (B-S): Buffer-size. **Consumers:**

```
while(true){
    while (counter == B-S)
        ; // do nothing.
    buffer[in] = nextProduced;
    in = (in+1) % B-S;
    counter++;
}
```

```
while(true){
    while (counter == 0)
        ; // do nothing
    nextConsumed = buffer[out];
    out = (out+1) % B-S;
    counter--;
}
```

⇒ counter ++:

```
register 1 = counter;
register 1 = register 1 + 1;
counter = register 1;
```

⇒ counter --:

```
register 2 = counter;
register 2 = register 2 - 1;
count = register 2;
```

⇒ Race Condition:

- Although both producer & consumer routines are correct separately, they may not function correctly when executed concurrently.
- Consider suppose the value of variable counter is currently 5. & that consumer & producer processes execute the statements "counter++" & "counter--" concurrently.

→ Following the execution of these 2 the value of variable counter may be 4, 5, or 6 but only correct result is counter = 5, this is obtained only when producer & consumer execute separately.

i.e S_0 : producer execute register 1 = counter.

→ register 1 = 5.

S_1 : producer execute register 1 + 1

→ register 1 = 6.

S_2 : consumer execute register 2 = counter

→ register 2 = 5.

S_3 : consumer execute register 2 - 1

→ register 2 = 4.

S_4 : producer execute counter = register 1

→ count = 6.

S_5 : consumer execute counter = register 2

→ count = 4.

→ Here we have arrived at the incorrect state "counter = 4" indicating that 4 buffers are full when in fact 5 buffers are full.

→ If we reverse the order of statements at S_4 & S_5 we arrive at incorrect state "counter = 6"

→ We would arrive at this incorrect state because we allowed both process to manipulate the variable counter concurrently.

→ A situation like this where several process access & manipulate the same data concurrently & the outcome of the execution depends on a particular order in which the access takes place is called a RACE CONDITION.

* Critical Section:

- Consider system of n processes $\{p_0, p_1, \dots, p_n\}$
- Each process has a segment of code called a critical section in which the process may be changing common variables, updating a table, writing a file & so on.
- When one process is in critical section no other process can be in its critical section.
- Critical section problem is to design a protocol that the process can use to cooperate.
- Each process must request permission to enter its critical section. In entry section may follow critical section with exit section then remainder section.

⇒ General structure of process p_i :

```

do {
    Entry section
    critical section.
    Exit section
    remainder section.
} while (TRUE);
  
```

⇒ Solution to Critical Section Problem:

- ① **Mutual Exclusion:** If process p_i is executing in its critical section, then no other processes can be executing in their critical section.
- ② **Progress:** If no process is executing in the critical section & there exists some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely.
- ③ **Bounded waiting:** A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section & before that request is granted.

Petersen's Solution:

- Software based solution to critical section problem.
- It is a 2 process solution that alternate execution between their critical section & remainder section. Numbered P_0 & P_1 processes.
- 2 data items required to be shared between 2 processes.

int turn;

boolean flag[2];

- Variable, turn indicates whose turn it is to enter its critical section.

Ex: if $turn = i$ means process P_i enters (CS).

- Flag array indicates if a process is ready to enter its critical section.

Ex: if $flag[i] = \text{True}$ then P_i ready to enter (CS).

- To enter the CS process P_i first sets the $flag[i]$ to be True & sets the turn to the value i , thereby telling that if another process wishes to enter critical section it can do so.

- If both processes try to enter critical section at same time, turn will be set to both i & j .

Structure of process in Petersen's solution:

⇒ P_i : do {
 $flag[i] = \text{TRUE};$
 $turn = i;$
 while ($flag[j] \& \& turn = j$);
 critical section.

$flag[i] = \text{FALSE};$
 remainder section.
 } while (TRUE);

⇒ P_j : do {
 $flag[j] = \text{TRUE};$
 $turn = j;$
 while ($flag[i] \& \& turn = i$);
 critical section.

$flag[j] = \text{FALSE};$
 remainder section.
 } while (TRUE);

- To prove that this solution is correct we need to show (3) ~~mutual~~ solution of critical section.

(i) mutual exclusion: each p_i enters its critical section only if either $\text{flag}[j] = \text{false}$ or $\text{turn} = i$.
 (ii) progress: to prove this solution must satisfy

(iii) Bounded waiting: we note that p_i can be prevented from entering the CS only if it is stuck in the while loop with condition $\text{flag}[j] = \text{true}$ & $\text{turn} = j$.

→ If $p[j]$ not read to enter critical section the $\text{flag}[j] = \text{false}$ & p_i can enter its critical section.

→ If p_j has set $\text{flag}[j]$ to true & is also executing in its while statement, then either $\text{turn} = j$ or $\text{turn} = i$ will enter critical section.

→ If $\text{turn} = i$ p_i will enter CS else $\text{turn} = j$ p_j will enter CS.

→ However once p_j exits its CS it will reset $\text{flag}[j]$ to false allowing p_i to enter its CS.

→ If p_j resets $\text{flag}[j]$ to true then it must also set $\text{turn} = i$.

→ Since p_i does not change the value of the variable turn while executing the while statement p_i will enter its CS.

* Semaphores:

→ It is a synchronization tool.

→ A semaphore S is an integer variable that apart from initialization is accessed only through 2 standard atomic operations: $\text{wait}(C)$ & $\text{signal}(C)$.

→ $\text{wait}(C)$ operation was originally termed P^n to test.

→ $\text{signal}(C)$ was originally called V to increment.

→ the definition of wait() is:

```
wait(s) {
    while (s <= 0)
        ; // no output.
}
```

→ the definition of signal() is:

```
signal(s) {
    s++;
}
```

→ All the modifications to the integer value of the semaphore in the wait() & signal() operations must be executed individually. i.e. when one process modifies the semaphore value, no other process can simultaneously modify the same semaphore value.

→ In case of wait(s) the testing of the integer value of s (s <= 0) & its possible modification (s--) must be executed without interruption.

⇒ Types of Semaphores:

① Binary Semaphore:

Also called as mutex semaphore.

→ It has only 2 values 0 & 1.

→ used to lock & unlock the access to resource.

→ Only 1 process can access critical section at a time.

② Counting Semaphore:

→ its value can range over unrestricted domain.

→ these are non-negative integers.

→ used to control access to a resource that has multiple instances.

→ the semaphore value indicates the number of available resources.

→ Semaphore provides mutual exclusion.

```
semaphore mutex;
do {
```

```
    wait(mutex);
```

```
    //critical section.
```

```
    signal(mutex);
```

```
    //remainder section.
```

```
} while (TRUE);
```

$S=0 \Rightarrow$ resource held by other process

$S=1 \Rightarrow$ resource is available.

Initial value $S=1$.

→ The value of counting semaphore can range over unrestricted domain for binary it is 0 & 1.

→ Each process that wishes to use a resource performs a `wait()` operation on the semaphore. (decrement the count).

→ Each process releases a resource, it performs a `signal()` operation (increment the count).

When count = 0 all resource are being used.

→ Semaphore Implementation:

→ Disadvantage of semaphore is that it require busy waiting, while a process is in its critical section any other process that tries to enter critical section must loop continuously in the entry code.

→ This will be problem in multiprogramming system.

→ Busy waiting wastes CPU cycle that some other process might be able to use.

→ To overcome this we modify definition of `wait()` & `signal()` semaphore operations.

→ When process executes `wait()` operation & finds that semaphore value is not ~~more~~ +ve it must wait.

→ But rather than engaging in busy waiting the process can block itself.

- No 2 process can execute wait CS & signal CS ^{same time}
- the block operation places a process into waiting queue associated with semaphore & the state of process is switched to waiting state.
- the control is transferred to CPU scheduler which selects another process to execute.
- A process that is blocked, waiting on a semaphore S should be restarted when some other process executes a signal CS operation.
- the process restarted by wakeup operation which changes the process from the waiting state to ready state.

Implementation of wait CS & signal CS!

wait (semaphore *S) {

S → value --;

if (S → value < 0) {

add this process to S → list;

block();

}

signal (semaphore *S) {

S → value ++;

if (S → value <= 0) {

remove process P from S → list;

wakeup(P);

}

Classic Problems of Synchronization!

① ~~Process~~ The Bounded Buffer problem!

- used to illustrate the power of synchronization primitives.
- we assume pool consists of n buffers each capable of holding one item
- the mutex semaphore provide mutual exclusion for access to buffer pool & is initialized to value

→ empty & full semaphore count the number of empty & full buffers.

semaphore empty $\rightarrow 1$.

semaphore full $\rightarrow 0$.

→ structure:

do {

// produce an item in nextp.

wait(empty);

wait(mutex);

// add item to buffer.

signal(mutex);

signal(full);

} while (TRUE);

~~producer~~
producer

do {

// ~~produce an item in nextp~~

wait(full);

wait(mutex);

// remove an item from buffer to nextc.

signal(mutex);

signal(empty);

// consume the item in nextc
} while (true);

consumer

② The readers-writers problem:

→ A database is to be stored among several concurrent ~~processes~~ processes.

→ Some may only want to read & some want to update the database.

→ structure of write process:

do {

wait(write);

// writing is performed;

signal(write);

} while (true);

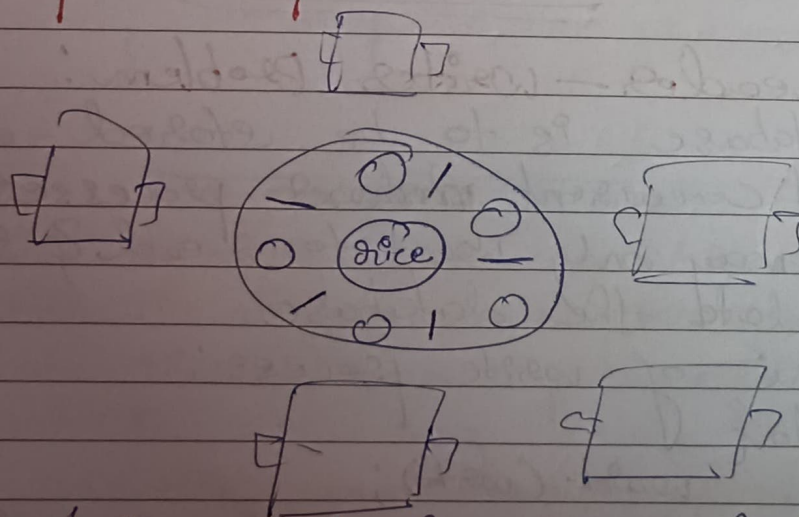
⇒ Structure for write process:

```
do {
    wait(mutex);
    readcount++;
    if (readcount == 1)
        wait(wrt);
    signal(mutex);
    // reading is performed.
    wait(mutex);
    readcount--;
    if (readcount == 0)
        signal(wrt);
} signal(mutex);
while (true);
```

⇒ Variations in reader-writer:

- (i) 1st variation: no reader kept waiting unless writer has permission to use shared output
 - (ii) 2nd variation: once writer is ready it performs write as soon as possible.
- Both have starvation leading to even more variation

③ Dining Philosophers Problem:



- Philosophers spend their lives thinking & eating
- Don't interact with their neighbors rationally
- try to pick up 2 chopsticks (one at time) to eat from bowl.

→ Need both chopstick to eat, then release both when done.

→ In case of 5 philosophers:
shared data:

Bowl of rice (data set)

Semaphore chopstick [5] initialized to 1.

→ structure of philosopher i:

do {

wait (chopstick[i]);

wait (chopstick[(i+1) % 5]);

// eat

signal (chopstick[i]);

signal (chopstick[(i+1) % 5]);

// think

} while (TRUE);

→ Problems with semaphore:

① Incorrect use of semaphore operations:

signal(mutex) - - wait(mutex)

wait(mutex) - - wait(mutex).

omitting of wait(mutex) or signal(mutex) or both.

② Deadlock & starvation.

③ Monitors: high level abstraction that provides a convenient & effective mechanism for process synchronization.

→ Only 1 process may be active within the monitor at a time.

→ Solution to Dining - Philosophers:

→ Each philosopher i invokes the operations pickup() & putdown() in following sequence:

DiningPhilosophers.pickup(i);

EAT

DiningPhilosophers.putdown(i);

→ No dead lock but starvation is possible.

monitor Dining Philosophers

```

{
    enum (THINKING; HUNGRY; EATING) state[5];
    condition self[5];
    void pickup(int i) {
        state[i] = HUNGRY;
        test(i);
    }
    if (state[i] != EATING) self[i].wait;

    void putdown(int i) {
        state[i] = THINKING;
        // test left & right neighbors
        test((i+4) % 5);
        test((i+1) % 5);
    }

    void test(int i) {
        if ((state[(i+4) % 5] != EATING) &&
            (state[i] == HUNGRY) &&
            (state[(i+1) % 5] != EATING)) {
            state[i] = EATING;
            self[i].signal();
        }
    }

    initialization code () {
        for (int i = 0; i < 5; i++)
            state[i] = THINKING;
    }
}

```

~~~~~

## ② DEAD LOCKS

### \* Deadlocks:

- A process requests the resources & if the resources are not available at that time the process enters a waiting state.
- Sometimes a waiting process is never again able to change state because the resources it has requested are held by other waiting processes.
- This situation is called deadlock.

### ⇒ Characteristics / Necessary conditions for DEADLOCK: 4 situations leading to deadlock:

① Mutual Exclusion: At least one resource is held in a non-sharable mode.

→ i.e. only one process at a time can use the resource.

→ If another process requests that resource, the requesting process must be delayed until resources are released.

② Hold & wait: A process must be holding at least one resource & waiting to acquire additional resource that are currently being held by other process.

③ No preemption: Resources cannot be preempted i.e. resource can be released only voluntarily by the process holding it, after that process has completed its task.

④ Circular wait: A set  $\{P_0, P_1, \dots, P_{n-1}\}$  of waiting processes must exist such that  $P_0$  is waiting for a resource held by  $P_1$ ,  $P_1$  is waiting for a resource held by  $P_2$ ,  $P_{n-1}$  is waiting for a resource held by  $P_n$  &  $P_n$  waiting for a resource held by  $P_0$ .

## \* Resource Allocation Graph:

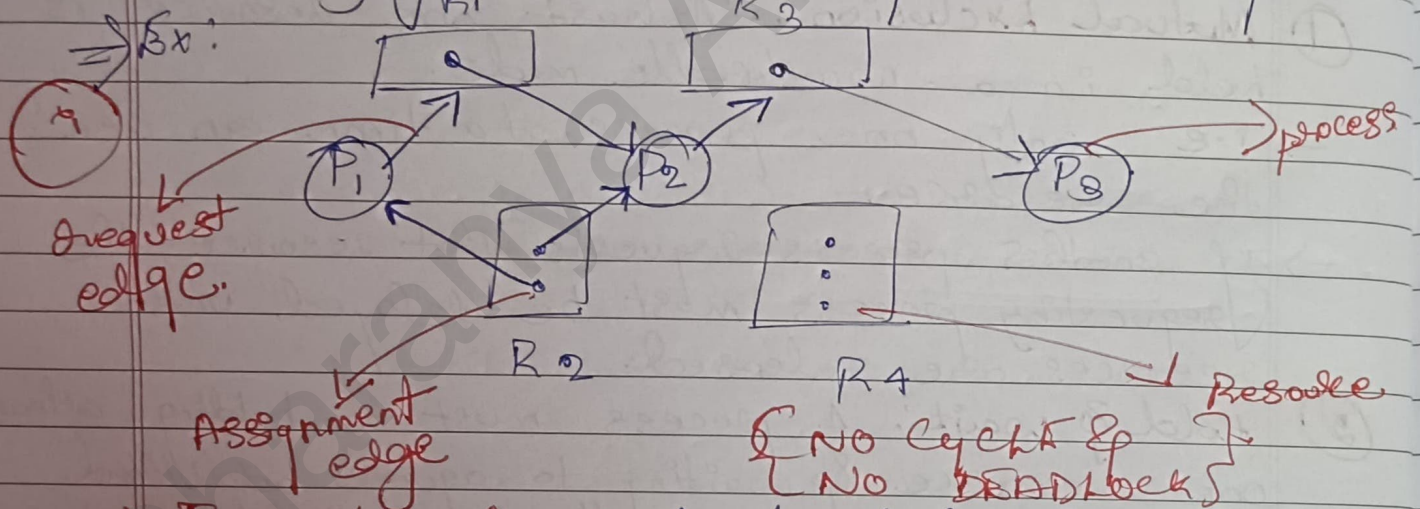
→ Deadlocks can be described more precisely in terms of a directed graph called as system resource allocation graph.

→ The graph contains set of vertices  $V$  & set of edges  $E$ .

→ The set of vertices  $V$  is partitioned in 2 different types of nodes:

$P = \{P_1, P_2, \dots, P_n\}$  the set consisting of all the active processes in the system &

$R = \{R_1, R_2, \dots, R_m\}$  set of resource types in the system.



→ **Request edge:** A directed edge from  $P_i \rightarrow R_j$ . It signifies that process  $P_i$  has requested an instance of resource type  $R_j$  & is currently waiting for the resource.

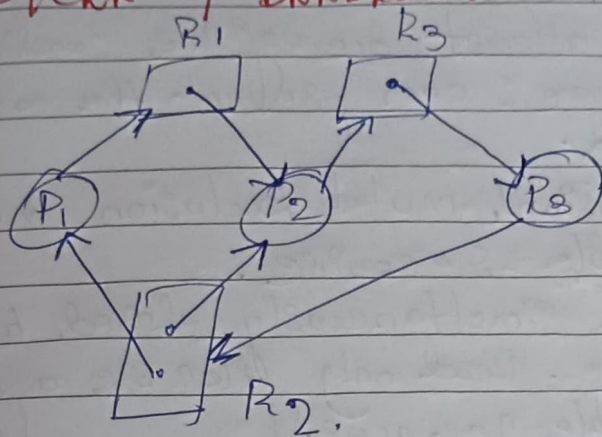
→ **Assignment edge:** A directed edge from  $R_j \rightarrow P_i$ . It signifies that an instance of resource type  $R_j$  has been allocated to process  $P_i$ .

→ **Representation:**

process :

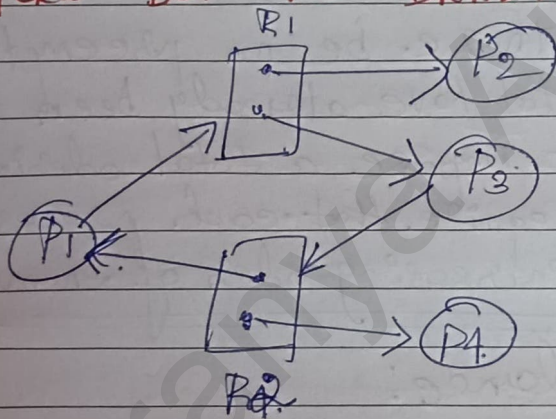
resource :

### Q.ii) CYCLE & DEADLOCK:



$P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_2$  is the cycle.  
& deadlock present.

### Q.iii) CYCLE BUT NO DEADLOCK:



$P_1 \rightarrow R_1 \rightarrow P_3 \rightarrow R_2 \rightarrow P_4 \rightarrow R_2 \rightarrow P_1$ .

### Note:

NO cycle  $\longrightarrow$  NO DEADLOCK.

IF Cycle:  $\longrightarrow$  one instance per resource Type: Deadlock  
may/may not  $\longrightarrow$  multiple instances per resource Type: NO Deadlock.

### \* Methods to handle Deadlock:

- Q.i) Use protocol to ensure that system will never enter a deadlock state.
- Q.ii) Allow system to enter a deadlock state & then recover.
- Q.iii) We ignore the problem altogether & pretend the deadlock never occurs in the system.

## \* Dead lock Prevention:

Ensuring that atleast one of the conditions cannot hold, we can prevent the occurrence of a deadlock.

(i) Mutual Exclusion: mutual Exclusion must hold for non shareable resource.

Ex: printer <sup>cannot</sup> be simultaneously shared by several process. Read only files are a good ex of a shareable resources.

(ii) Hold & wait: To ensure that hold & wait never occur in the system we must guarantee that whenever a process requests a resource, it does not any other resources.

(iii) No preemption: There be no preemption for the resources that have already been allocated.

(iv) Circular wait: Impose a total ordering of all resource & require that each process request resources in an increasing order of enumeration.

## \* Deadlock Avoidance:

\* Safe State: A state is safe if the system ~~not~~ can allocate resource to each process in some order & still avoid a deadlock.

→ System in safe state only if there exist safe sequence.

→ A deadlock state is an unsafe state.

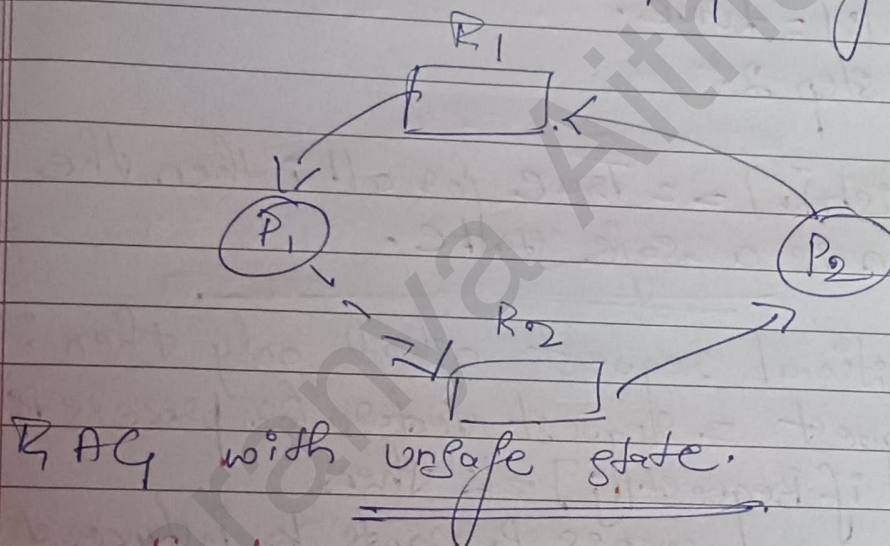
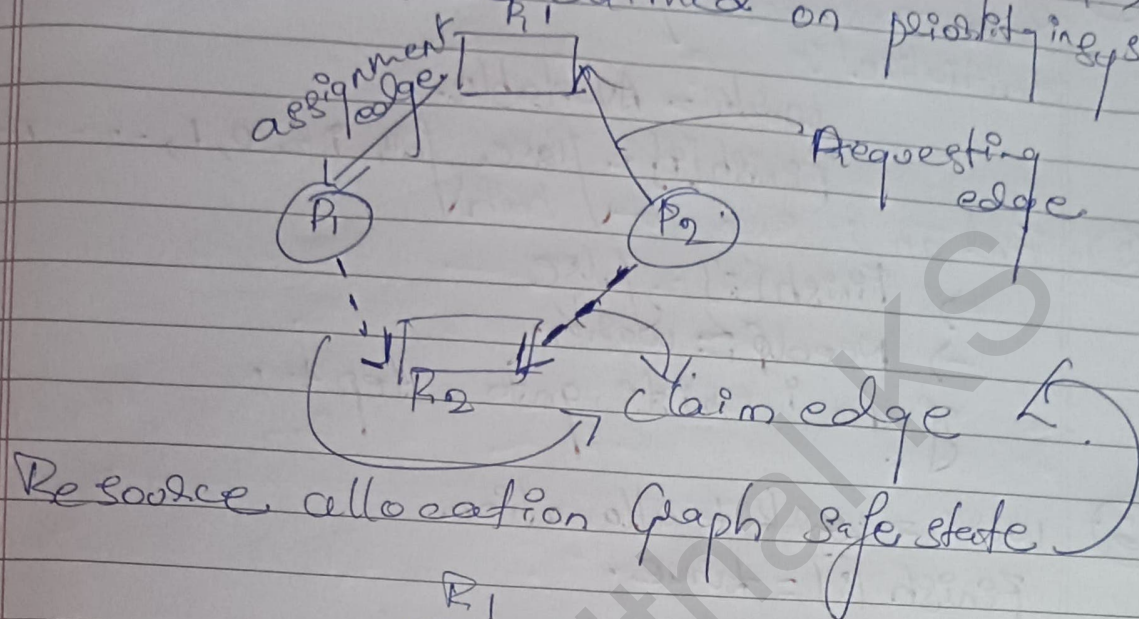
→ Not all unsafe state is deadlocked.

## \* Resource Allocation Graph Algorithm:

→ Claim Edge:  $P_i \rightarrow R_j$  indicated that process  $P_i$  may request resource  $R_j$  represented by dashed line.

→ When process  $P_i$  request  $R_j$  the claim edge  $P_i \rightarrow R_j$  converted to request edge.

- When  $R_j$  is released by  $P_i$  the assignment edge  $R_j \rightarrow P_i$  is reconverted to claim edge  $P_i \rightarrow R_j$ .
- Resource must be claimed on priority system.



## \* Banker's Algorithm:

### ⇒ Data Structures:

Let  $n$  = number of processes &  $m$  : no of resource type

① **Available**: Vector of length  $m$ . If  $available[j] = k$  there are  $k$  instances of resource type  $R_j$  available.

② **Max**:  $n \times m$  matrix. If  $max[i, j] = k$  then process  $P_i$  may request at most  $k$  instance of resource  $R_j$ .

③ **Allocation**:  $n \times m$  matrix, If  $allocation[i, j] = k$  then  $P_i$  currently allocated  $k$  instances of  $R_j$ .

④ **Need**:  $n \times m$  matrix. If  $Need[i, j] = k$  then  $P_i$  may need  $k$  more instances of  $R_j$  to complete its task.  
 $Need[i, j] = Max[i, j] - Allocation[i, j]$

## Safe Algorithm:

- ① Let  $work$  &  $Finish$  be vectors of length  $m$  &  $n$ , respectively.  
Initialize:

$work = Available.$

$Finish[i] = false$  for  $i = 0, 1, \dots, n-1.$

- ② Find an  $i$  such that both  
 $\rightarrow Finish[i] = false$   
 $\rightarrow Need_i \leq work.$   
 if no  $i$  exists goto step 4.

- ③  $work = work + allocation$   
 $Finish[i] = true.$   
 go to step 2

- ④ if  $Finish[i] = true$  for all  $i$  then the system is a safe state.

{if additional request asked only then :}  
 $Request = request\ vector\ for\ process\ P_i.$   
 if  $Request[i] = k$  then  
 process  $P_i$  wants  $k$  instance of resource type  $P_i$

- ① if  $Request \leq Need_i$  goto step 2  
 else raise error condition, since process exceeded maximum claim
- ② if  $Request_i \leq Available$ , goto step 3.  
 else  $P_i$  must wait since resource not available
- ③ Proceed to allocate requested resource to  $P_i$  by modifying state as follows:  
 $Available = Available - Request_i;$   
 $Allocation_i = Allocation_i + Request_i;$   
 $Need_i = Need_i - Request_i;$

Safe — resource allocated to  $P_i$   
 Unsafe —  $P_i$  must wait & old resource allocation state is restored

| ① Process      | Allocation |   |   |   | MAX |   |   |   | Available |   |   |   |
|----------------|------------|---|---|---|-----|---|---|---|-----------|---|---|---|
|                | A          | B | C | D | A   | B | C | D | A         | B | C | D |
| P <sub>0</sub> | 0          | 0 | 1 | 2 | 0   | 0 | 1 | 2 | 1         | 5 | 2 | 0 |
| P <sub>1</sub> | 1          | 0 | 0 | 0 | 1   | 7 | 5 | 0 |           |   |   |   |
| P <sub>2</sub> | 1          | 3 | 5 | 4 | 2   | 3 | 5 | 6 |           |   |   |   |
| P <sub>3</sub> | 0          | 6 | 3 | 2 | 0   | 6 | 5 | 2 |           |   |   |   |
| P <sub>4</sub> | 0          | 0 | 1 | 4 | 0   | 6 | 5 | 6 |           |   |   |   |

What is content of need matrix?

Is the system in safe state?

If a request from process P<sub>1</sub> arrives for 0 4 2 0. Can request be granted immediately?

① need matrix:  $\text{max} - \text{Allocation}$ .

| Process        | A | B | C | D |
|----------------|---|---|---|---|
| P <sub>0</sub> | 0 | 0 | 0 | 0 |
| P <sub>1</sub> | 0 | 7 | 5 | 0 |
| P <sub>2</sub> | 1 | 0 | 0 | 2 |
| P <sub>3</sub> | 0 | 0 | 2 | 0 |
| P <sub>4</sub> | 0 | 6 | 4 | 2 |

② Safe sequence:

①  $\text{need} \leq \text{Available}$ .

{ T : New available }  
{ F : No update }

$$P_0: 0000 \leq 1520.$$

TRUE

$$\text{new available} = \text{old available} + \text{Allocation}$$

$$= 1520 + 0012$$

$$= 1532$$

$$P_1: 0750 \leq 1532 \leftarrow$$

FALSE

$$P_2: 1002 \leq 1532$$

TRUE

$$NA = 0A + \text{Allocation}$$

$$= 1532 + 1354$$

$$= 2886$$

$$P_3: 0020 \leq 2886$$

TRUE

$$NA: 2886 + 0632 = 214118$$

$$P_4: 0642 \leq 2 \ 14 \ 11 \ 8$$

TRUE

$$NA: 2 \ 14 \ 11 \ 8 + 0014 \\ = 2 \ 14 \ 12 \ 12$$

$$P_1: 0750 \leq 2 \ 14 \ 12 \ 12$$

TRUE

$$NA: 2 \ 14 \ 12 \ 12 + 1000 \\ 3 \ 14 \ 12 \ 12$$

$\therefore$  Safe sequence:  $\langle P_0, P_2, P_3, P_4, P_1 \rangle$

9.ii) Additional Request:  $P_1 \langle 0420 \rangle$

Request  $\leq$  Need

$$0420 \leq 0750$$

TRUE

$$\text{new Available} = \text{old available} - \text{request} \\ = 1520 - 0420 \\ = 1100$$

$$\text{new Allocation} = \text{old allocation} + \text{request} \\ = 1000 + 0420 \\ = 1420$$

$$\text{new Need} = \text{old need} - \text{request} \\ = 0750 - 0420 \\ = 0330$$

Updated table:

| Process        | Allocation |   |   |   | max |   |   |   | Available |   |   |   | Need |   |   |   |
|----------------|------------|---|---|---|-----|---|---|---|-----------|---|---|---|------|---|---|---|
|                | A          | B | C | D | A   | B | C | D | A         | B | C | D | A    | B | C | D |
| P <sub>0</sub> | 0          | 0 | 1 | 2 | 0   | 0 | 1 | 2 | 1         | 1 | 0 | 0 | 0    | 0 | 0 | 0 |
| P <sub>1</sub> | 1          | 4 | 2 | 0 | 1   | 7 | 5 | 0 |           |   |   |   | 0    | 3 | 3 | 0 |
| P <sub>2</sub> | 1          | 3 | 5 | 4 | 2   | 3 | 5 | 6 |           |   |   |   | 1    | 0 | 0 | 2 |
| P <sub>3</sub> | 0          | 6 | 3 | 2 | 0   | 6 | 5 | 2 |           |   |   |   | 0    | 0 | 2 | 0 |
| P <sub>4</sub> | 0          | 0 | 1 | 4 | 0   | 6 | 5 | 6 |           |   |   |   | 0    | 6 | 4 | 2 |

⇒ Again check for Safe Sequence!

$P_0$ : need  $\leq$  available

$$0000 \leq 1100 \quad \text{TRUE}$$

$$NA: 1100 + 0012 \\ = 1112.$$

$$P_1: 0330 \leq 1112 \quad \text{FALSE}$$

$$P_2: \cancel{0330} 1002 \leq 1112 \quad \text{TRUE}$$

$$NA: 1112 + 1354 \\ 2466.$$

$$P_3: 0020 \leq 2466 \quad \text{TRUE}$$

$$NA: 2466 + 0632 \\ 21098.$$

$$P_4: 0642 \leq 21098 \quad \text{TRUE}$$

$$NA: 21098 + 0014 \\ 2101012.$$

$$P_1: 0330 \leq 2101012 \quad \text{TRUE}$$

$$NA: 2101012 + 1420 \\ 3141212.$$

∴ Safe Sequence!  $\langle P_0, P_2, P_3, P_4, P_1 \rangle$ .

the Same Sequence for the given problem.  
 $P_0, P_2, P_3, P_4, P_1$ .

Since it is in Safe Sequence the request can be granted immediately.

**\* NOTE:**

① Need Matrix  $\Rightarrow$  max - Allocation

② need  $\leq$  Available.

$$\text{new Available} = \text{old available} + \text{Allocation}$$

③ Request  $\leq$  Need

$$\text{new Available} = \text{available} - \text{request}$$

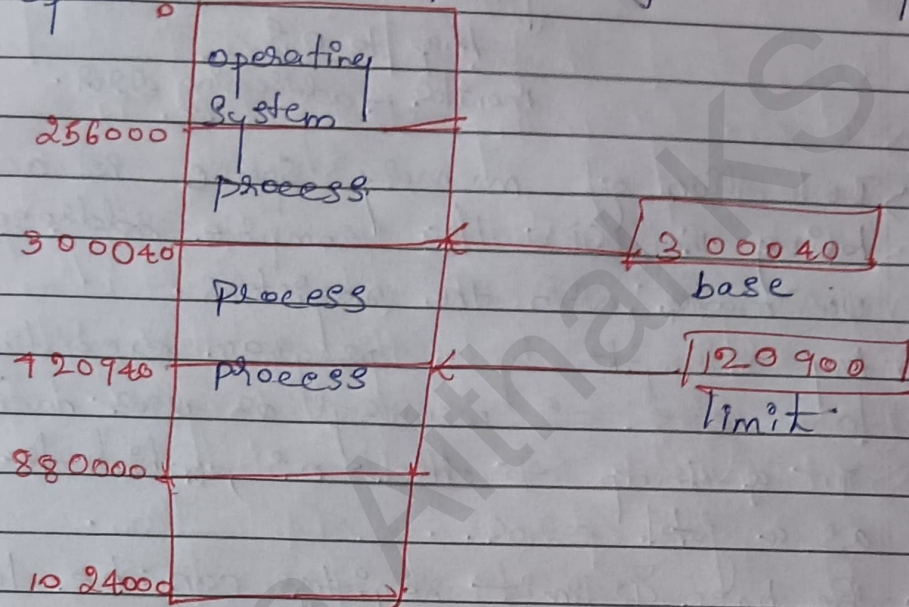
$$\text{new Allocation} = \text{Allocation} + \text{request}$$

$$\text{new need} = \text{need} - \text{request}$$

## ③ MAIN MEMORY

### \* Base & Limit Register:

→ Pair of base & limit registers define the logical address space.



→ We must ensure correct operation has to protect the operating system from access by user processes & in addition protect user processes from one another. This protection must be provided by hardware.

→ We first need that each process has separate memory space, for this we need ability to determine range of legal addresses that the process may access only these legal addresses.

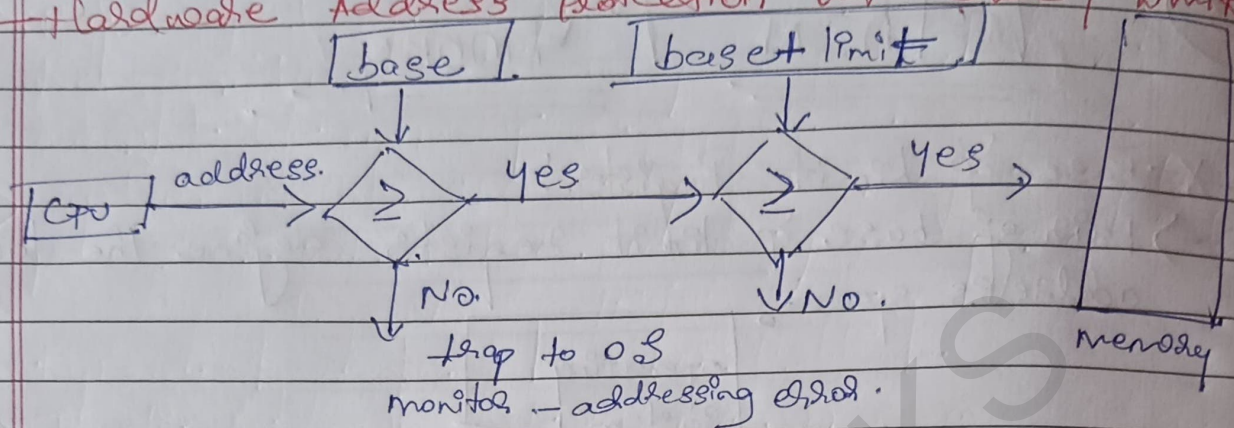
→ We provide the protection using 2 registers a BASE & LIMIT registers.

→ LIMIT register specifies size of the range.

→ BASE register holds smallest legal physical memory address.

Ex: If limit register specifies the size of the range 1500 then if base register holds 1600 then the program can legally access all addresses from 1600 through 3100 (inclusive).

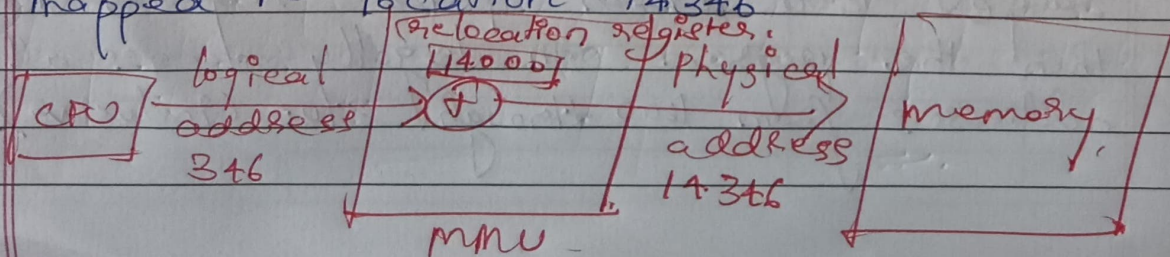
## \* Hardware Address Protection with Base & Limit:



- Protection of memory space is accomplished by having the CPU b/w compare address generated by user mode with the registers.
- Any attempt by a program executing in user mode to access OS memory or user memory results in a trap to the OS, which treats the attempt as a fatal error.
- The base & limit registers can be loaded by the OS which uses a special privileged instruction i.e. kernel mode.

## \* Memory Management Unit (MMU):

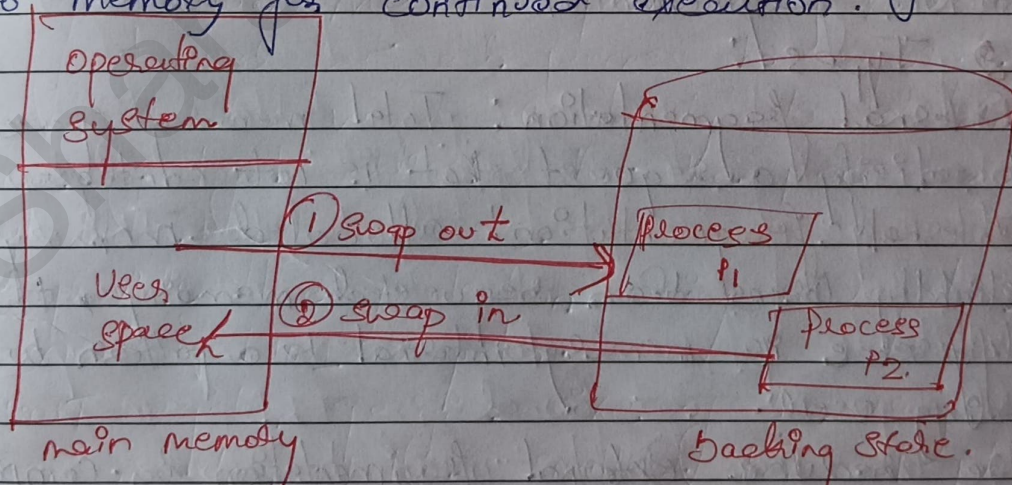
- The run time mapping from virtual to physical addresses is done by hardware device called MMU.
- The base register now called a relocation register.
- The value in this is added to every address generated by a user process at the time. It is sent to memory.
- Ex: If the base is at 14000 then an attempt by the user to address locator 0 is dynamically reallocated to location 346. 0 is ~~attempted to~~ mapped to location 14346.



- Dynamic relocation using a relocation registers:
- To obtain better memory space utilization we use dynamic loading.
  - Here a routine is not loaded until it is called.
  - All routines are kept on disk in a relocatable load format.
  - Main program is loaded into memory & is executed.
  - When a routine needs to call another routine, the calling routine first checks to see whether the other routine has been loaded.
  - If not the relocatable linking loader is called to load the desired routine into memory & to update the program address table to reflect this change. Then control is passed to newly loaded routine.

### \* Swapping:

- A process can be swapped temporarily out of memory to a backing store & then brought back into memory for continued execution.



- Backing store: Just disk large enough to accommodate copies of all memory images for all users, must provide direct access to these memory images.
- Roll out & roll in: swapping variant used for priority based scheduling algorithms, lower priority process is swapped out so higher one can be loaded & executed.

- major part of swap is transfer time, total transfer time is directly proportional to the amount of memory swapped.
- System maintains a ready queue of ready to run processes which have memory images on disk.

### \* Dynamic Storage Allocation Problem:

- To satisfy a request of size,  $n$  from a list of free holes.
- Holes: Available free memory space:

  - ① First fit: Allocate the first hole that is big enough.
  - ② Best fit: Allocate the smallest hole that is big enough.
  - ③ Worst fit: Allocate the largest hole.

- 1st & Best fit better than worst fit in terms of speed & storage utilization.

### \* Fragmentation:

- A process is loaded/removed from the memory the free memory space is broken into little pieces that is fragmentation.

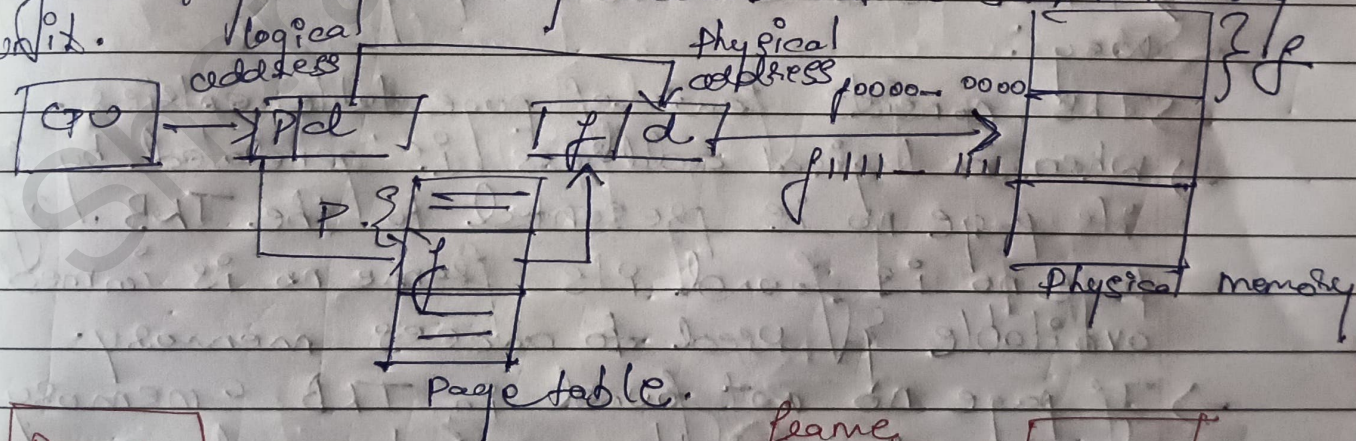
#### 2 Types:

- ① External fragmentation: Total memory space exists to satisfy a request that is not contiguous.
- ② Internal fragmentation: allocated memory can be slightly larger than requested memory, this size difference is memory internal to a pattern but not being used.

- We reduce external fragmentation by compaction.
- Compaction is possible only if deallocation is dynamic & is done at execution time.
- plan is to place all free memory together in one large block by shuffling the memory contents.

## \* Paging:

- It is a memory management scheme that provides the physical addresses space of a process to be non contiguous.
- Basic method:
  - Break physical memory into fixed sized blocks called frames. & break logical memory into fixed blocks of same size called pages.
  - When a process is to be executed its pages are loaded into any available memory frames from the backing store, which is divided into fixed sized blocks that are of the same size as the memory frames.
  - Every address generated by CPU is divided into 2 parts:  $\text{pageno. (p)}$  &  $\text{page offset (d)}$
  - Page No is used as 'index' into a page table
  - Page table contains base address of each page in physical memory.
  - This base address is combined with offset to define physical memory address that is sent to memory unit.



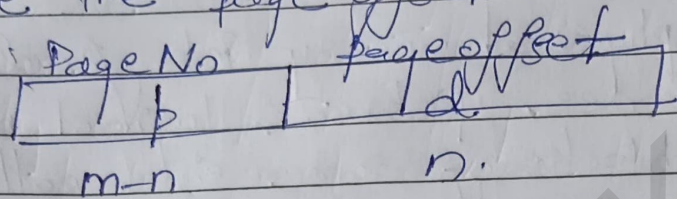
| Page 0         |   |   |
|----------------|---|---|
| Page 1         | 0 | 1 |
| Page 2         | 1 | 4 |
| Page 3         | 2 | 3 |
| Logical memory | 3 | 7 |

page table

| frame numbers 0 |        |
|-----------------|--------|
| 1               | Page 0 |
| 2               |        |
| 3               | Page 2 |
| 4               | Page 1 |
| 5               |        |
| 6               | Page 3 |
| 7               |        |

Physical memory

→ If size of logical address space is  $2^m$  & page size is  $2^n$  addressing units, then high-order  $m-n$  bit of a logical address designate the page no & the  $n$  low order bit designate the page offset.



where  $p$  is index into the page table &  $d$  is the displacement within the page.

### \* Paging Hardware with TLB

#### [Translation Lookaside Buffer]

→ TLB is a special small fast lookup hardware cache that is associative & high speed memory.

→ Each entry in TLB consists of 2 parts:  
→ a key (or tag)  
→ a value.

⇒ TLB is used with page table in following way:

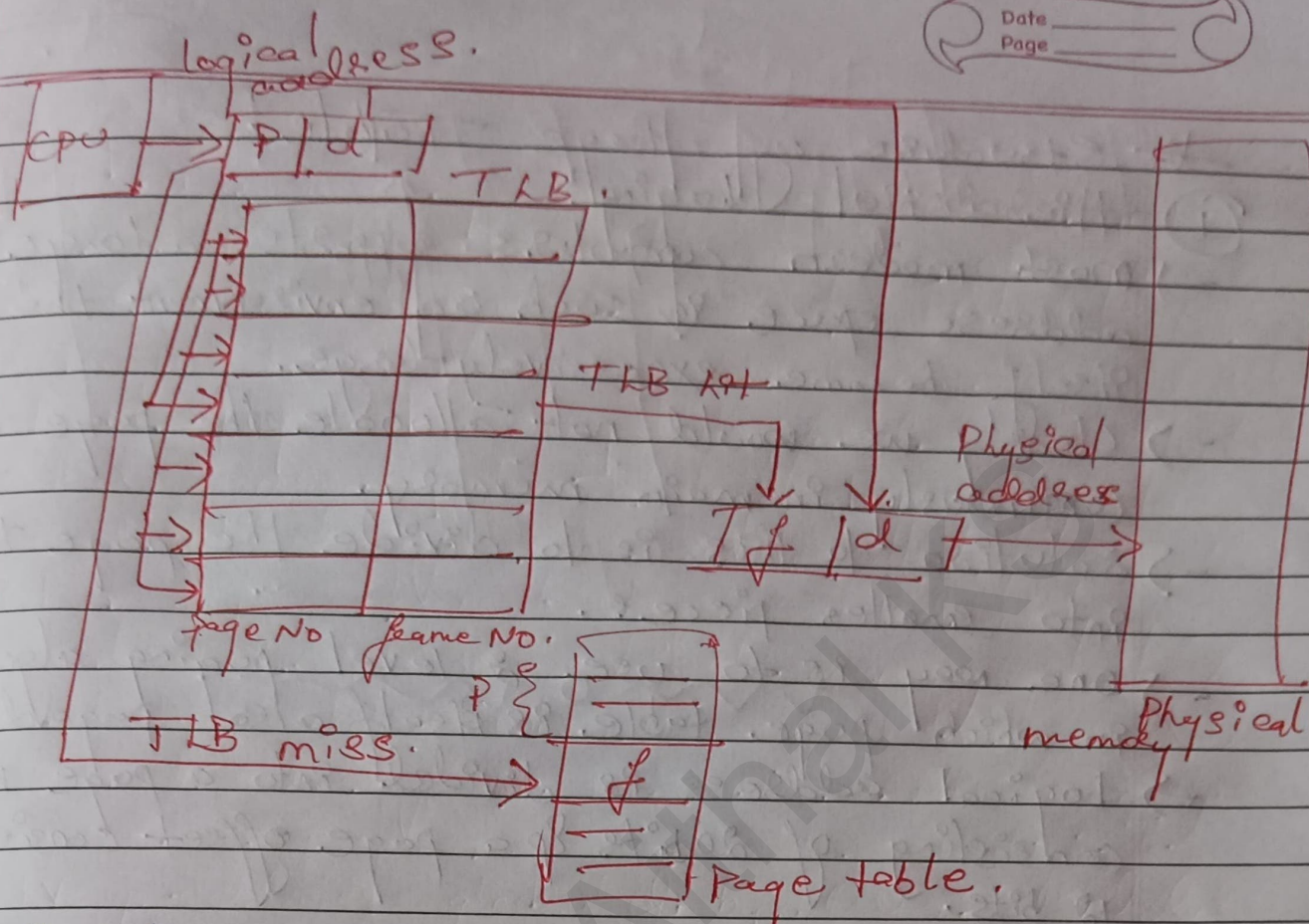
→ TLB consists few of page table entries.

→ When a logical address is generated by CPU, its page no is presented to the TLB. If the page no is found its frame no is immediately available & used to address memory.

→ If page no not found in TLB a memory reference to the page table must be made.

→ When the frame no is obtained, we can use it to access memory; also we add page no & frame no to the TLB, so that they will be found quickly on the next reference.

→ If TLB full then page replacement (LRU)



→ The % of times that a particular page no is found in the TLB is called hit ratio.

### \* Valid - Invalid Bit:

→ These are attached to each entry in the page table.

→ Valid: indicates that the associated page is in the process' logical address space & is thus a legal page.

→ Invalid indicates that the page is not in the process logical address space.

→ Any violations results in a trap to the kernel.

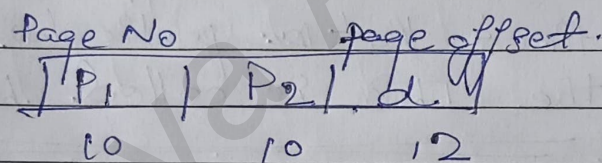
| Logical Address | Page No | frame No | Valid/Invalid bit | Physical Address |
|-----------------|---------|----------|-------------------|------------------|
| 0000            | page 0  | 0        | V                 | 0                |
|                 | page 1  | 1        | V                 | 1                |
|                 | page 2  | 2        | V                 | 2                |
|                 | page 3  | 3        | V                 | 3                |
|                 | page 4  | 4        | V                 | 4                |
|                 | page 5  | 5        | V                 | 5                |
| 10468           |         | 6        | I                 |                  |
| 12287           |         | 7        | I                 |                  |

Page table

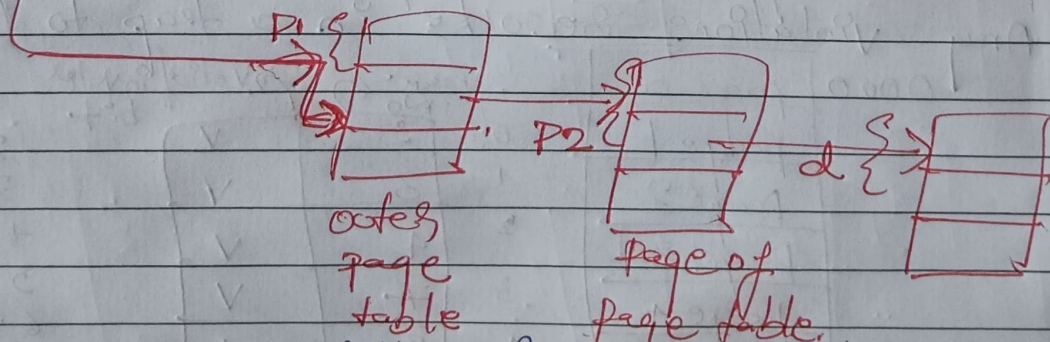
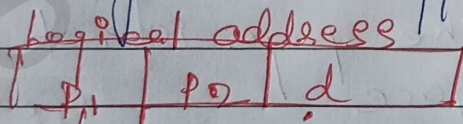
## \* Structure of Page Table:

### ① Hierarchical (Paging):

- most modern computers supports a large logical address space. In such an environment the page table itself become excessively large.
- Clearly we would not allocate the page table continuously in main memory.
- Solution to this is to divide the page table into smaller pieces.
- One way is to use 2 level paging algorithm in which page table is itself also paged.
- Logical address is divided into a page no consisting of 20 bits & a page offset consisting of 12 bits.



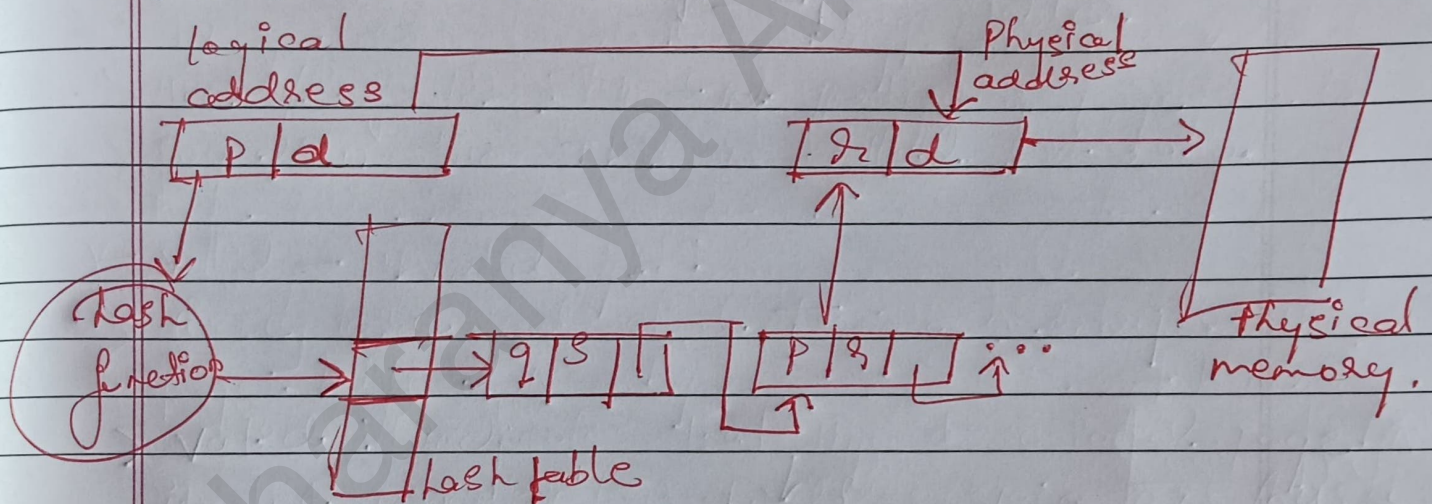
- where  $P_1$  is an index into outer page table &  $P_2$  is the displacement within the page of the outer page table.
- Because address translation works from the outer page inward, this scheme is also known as "forward mapping" <sup>page</sup>table.



Address translation for a 2 level 32 bit paging architecture

## ② Hashed Page Table:

- A common approach for handling address space.
- Address space larger than 32 bits is to use a hashed page table, with the hashed value being the virtual page no.
- Each entry in the hash table contains a linked list of elements that hash to the same location.
- Each element consists of a 3 fields
  - ① Virtual page no
  - ② Value of the mapped page frame.
  - ③ A pointer to the next element in the linked list
- Algorithm works as follows: the virtual page no in the virtual address is hashed into the hash table



## ③ Inverted Page Table:

- Draw back of hashed page table is that each page table may consist of millions of entries.
- To solve this we use inverted page tables.
- An inverted page table has one entry for each real page of memory.
- Each entry consists of the virtual address of the page stored in that real memory location with information about the process that owns the page.
- Each virtual address in the system consists of a triple  $\langle \text{process-id, page no \& offset} \rangle$ .

